KU LEUVEN

FACULTY OF ENGINEERING TECHNOLOGY

KU LEUVEN - BRUGES

Modeling object localization uncertainty for robot grasping

Thijs ALENS

Supervisor(s): Prof Dr Mathias Verbeke
Dr Ir Laurens Devos

Master's Thesis submitted to obtain the degree of Master of Science in Artificial Intelligence in Business and Industry

Academic year 2025 - 2026

©Copyright KU Leuven

This master's thesis is an examination document that has not been corrected for any errors.

Without written permission of the supervisor(s) and the author(s) it is forbidden to reproduce or adapt in any form or by any means any part of this publication. Requests for obtaining the right to reproduce or utilise parts of this publication should be addressed to KU Leuven - Bruges, Spoorwegstraat 12, B-8200 Bruges, +32 50 66 48 00 or via e-mail fet.bruges@kuleuven.be.

A written permission of the supervisor(s) is also required to use the methods, products, schematics and programs described in this work for industrial or commercial use, for referring to this work in publications, and for submitting this publication in scientific contests.

Preface

Het voorwoord vul je persoonlijk in met een appreciatie of dankbetuiging aan de mensen die je hebben bijgestaan tijdens het verwezenlijken van je masterproef en je hebben gesteund tijdens je studie.

In the preface, you write a personal account with an acknowledgement or expression of gratitude to the people who assisted you in completing your master's thesis and supported you during your studies.

Copyright Information:
student paper as part of an academic education and examination.
No correction was made to the paper after examination.

Samenvatting

De (korte) samenvatting, toegankelijk voor een breed publiek, wordt in het Nederlands geschreven en bevat **maximum 3500 tekens**. Deze samenvatting moet ook verplicht opgeladen worden in KU Loket.

Summary

This (short) summary, accessible to a wide audience, is written in English and contains a **maximum of 3500 characters**. This summary must also be uploaded to KU Loket.

Keywords: Enter (more or less) five keywords.

Contents

1 Introduction **2**

1.1 Positioning 2

1.2 Problem statement 2

1.3 Objectives 3

2 Literature study **5**

2.1 Uncertainty in AI 5

2.1.1 Aleatoric uncertainty 5

2.1.2 Epistemic uncertainty 6

2.1.3 Sources of uncertainty 7

2.2 Quantifying uncertainty 8

2.2.1 Aleatoric uncertainty 8

2.2.2 Epistemic uncertainty 8

2.3 Uncertainty-aware AI models 9

2.3.1 Bayesian Neural Networks 9

2.3.2 Deep Ensemble Methods 11

2.3.3 Monte Carlo dropout 12

3 Dataset **14**

3.1 Setup 14

3.1.1 Realsense camera 14

3.1.2 Table and robot arm 14

3.2 Labeling 16

3.2.1 Pixel coordinates 16

3.2.2 Real-world coordinates 18

3.2.3 Robot's position 19

3.3 Capturing and using data 19

3.3.1 Collecting data 19
3.3.2 PyTorch Dataset 20

4 Models for uncertainty aware object detection 22

5 Safety mechanisms for uncertain predictions 23

6 Conclusion 24
6.1 Future work 24

7 GenAI code of conduct for students 27

List of Figures

2.1 The blue line represents a model which is not well generalized, and thus captures the noise in the data. The red line represents a model which is well generalized, and thus captures the general distribution of the data. 9

3.1 The setup used. 15

3.2 Annotating the pixel coordinates of the objects. 17

3.3 The chessboard pattern used for calibration. 18

3.4 An example of a depth image with its corresponding RGB image captured by the Realsense camera. 21

List of Tables

Acronyms

AI artificial intelligence. 2, 3, 5, 7, 9, 18, 19

CNN convolutional neural network. 5, 6, 9

CV computer vision. 2

DNN deep neural network. 5, 6, 11

GAN generative adversarial network. 7, 9

MC Monte Carlo. 8, 13

MLP multilayer perceptron. 6, 9

NN neural network. 9, 11, 12

PoV point of view. 16

Chapter 1

Introduction

1.1 Positioning

We are living in a world where automation is adopted at a rapidly increasing pace. Luggage gets sorted and autonomously transported across an airport, autonomous vacuum cleaners are being used daily, medicines are being produced automatically with extreme cleanliness, etc. This also applies to robotics, where robots are slowly becoming more apparent in factory's and warehouses, improving efficiency, quality, and safety.

These robots need systems to operate properly, now more then ever, these systems are often based on artificial intelligence (AI). More precisely, computer vision (CV), a subdomain of AI which specialises in enabling computers to interpret and make decisions based on visual data via camera's, can be used to detect and localize objects, making it possible for the robot to process them. But also a more general AI could be used to plan the robot's actions and control its actuators to perform the desired tasks. Furthermore, some systems are trained to monitor certain processes, detect defects to finished products, or even predict maintenance needs. Its use thus does not only bring more efficiency, but also some gains in quality and enables for much more complex tasks for a robot to excecute.

These real world scenarios require the AI systems to be robust and reliable. However, AI systems often struggle when faced with new, unknown situations, leading to unreliable predictions. Systems should therefore be able to express their uncertainty about their predictions, enabling safety mechanisms to prevent the robot from taking actions based on uncertain predictions. Uncertainty-aware AI models are able to do this, by not only outputting a prediction, but also an uncertainty score about this prediction.

1.2 Problem statement

While an AI system has many advantages in a factory setting, it also introduces new problems that need taken care of. One of the issues with these systems is that they will always give

you an answer, even when they are unsure about it. Meaning that when it were to be unsure about what it should output, it will not inform the user about this and continue operating as normal. This could be good, because it will sometimes be able to make the right decision even when unsure, increasing efficiency compared to a human worker which would need to inspect and learn the new situation. But it will most probably be bad, since it can do things wrong, which could lead to defects to the products, damage to the robot or its environment, or even worse, harm to humans working nearby.

One example of why a system could be unsure about its output is when it needs to infer something from data that is not seen (properly) during training. Most training data is gathered in controlled environments, where all conditions are optimal. However, when used in real-world scenarios with conditions different from the training environment, like different lighting conditions because of shadows or reflections, some dust on the camera lens causing some distortion, someone turning the lights off, etc. the system could become a lot worese at its task. Other unexpected events, like a human operator placing the wrong object for the robot to pick up or a sudden movement of an already placed object, can also throw of the AI system.

1.3 Objectives

This thesis aims to adress these issues by developing and evaluating uncertainty-aware object localization models for a robot arm capable of reliably indicating when their predictions may be unreliable, particularly in challenging real-world conditions. By making the models uncertainty-aware, safety mechanisms that prevent the robot from taking actions based on uncertain predictions can be implemented, thereby reducing the risk of errors and accidents.

The primary research question is: How can we develop uncertainty-aware object localization models for a robot arm that can effectively identify and communicate uncertainty in their predictions, particularly in challenging real-world conditions?

This overarching question is further explored through the following sub-questions:

- What uncertainty aware (object localization) models already exist, and how do they perform in real-world conditions?
- How can we effectively quantify and communicate uncertainty in object localization predictions to ensure safe robot operation?

To address these questions, the following objectives are set:

- Conduct a comprehensive literature review on uncertainty and existing uncertainty-aware models with an emphesis on object localization.
- Create a dataset that simulates real-world conditions, including various challenges such as occlusions, lighting variations, unexpected object placements, etc.

- Develop, implement and evaluate uncertainty-aware object localization models for our use case.
- Design and implement safety mechanisms that utilize the uncertainty information to prevent the robot from taking actions based on uncertain predictions.

Chapter 2

Literature study

In this chapter, we discuss existing research relevant to uncertainty in AI models and how it can be mapped to real-world scenarios. First, we will elaborate on what uncertainty is and different ways it can sneak in AI models. Next, we will discuss how this uncertainty can be quantified. After that, we will dive into different existing methods to make AI models uncertainty-aware and discuss how their ideas can be applied to object detection. Following that, we will look into ways data is gathered to train and evaluate uncertainty-aware models.

2.1 Uncertainty in AI

There are lots of complex systems that need to be modeled and understood. These systems can be modelled using AI models as they offer a way to grasp the complexity of these systems and operate them in the most efficient / safe way possible. However, these are, just as humans, not perfect and can make mistakes. When using normal (non uncertainty-aware) AI models, these mistakes are not communicated with the user. For example, when a convolutional neural network (CNN) is used to detect an object on a conveyor belt, it will output the class of the object it thinks is on the conveyor belt, without any indication of how certain it is about this prediction. So if the object is only partially visible, the lighting is bad, or the object is very similar to another object, the model might be very uncertain about its prediction, but it will still output a class as if it is certain about it.

All possible uncertainties regarding deep neural networks (DNNs) can be captured in two groups: aleatoric uncertainty and epistemic uncertainty [1] [2].

2.1.1 Aleatoric uncertainty

Aleatoric uncertainty is the uncertainty that comes from noise in the data. Its nature is irreducible, meaning that no matter how much data we gather, this uncertainty will always be present in the data.

Measurement errors When gathering data, mistakes can be made when measuring. These can come from limitations of the measurement devices or incorrect / inprecise labeling of the data. An important note to make is that these mistakes can be a way to regulize a network, if it stays subtle.

As an example, we will use the regression problem we are trying to solve in this thesis where a model needs to predict a pair of x and y values for the center of an object. It could be that the camera used for the capturing of data has a low resolution. This could lead to an object, which needed to be detected, getting lost because of the lack of pixels in the image. Even more, it could lead to inaccurate annotation of the center of the object, as its center is labeled as (x_1, y_1), when it is in fact $(x_1 + \delta, y_1 + \epsilon)$ (δ and ϵ being small errors in the measurement).

2.1.2 Epistemic uncertainty

Epistemic uncertainty is the uncertainty that comes from noise in the model or its training process. This can have many different causes like the architecture of the model, the training procedure, lack of knowledge because of insufficient or irrelevant training data. Its nature is, in theory, reducible, meaning that by gathering more / better data, using a better model architecture, etc., this uncertainty can be reduced. However, doing this in practice, comes with a big computational cost, and thus it is not always possible to reduce this uncertainty.

Variability in real world situations The real world changes constantly, resulting in different settings where objects can behave / look differently. All situations should be covered sufficiently by the training set. If not, the network trained on the data can not garentee a good output for every new real world situation. A distribition shift occurs when real world situations differ from the training set. These shifts are very hard for a DNN to grasp, causing its performence to change significantly.

Continueing the example of of the regression problem where a model needs to predict the center of an object. When a DNN is trained on a dataset which covers the objects with great lighting, no occlusions, no other objects covering the view, etc., it will be very good at predicting the center of the object in that specific setting. However, it might happen that, when using the model, the conditions change (e.g., lighting changes, new objects appear, etc.). Because the dataset does not cover the new conditions, the trained DNN fails to addapt to these new conditions, causing its performence to drop.

Architecture specification errors The structure of a network differs from model to model, with each model having its advantages and disadvantages on the performence, and thus the uncertainty of the predictions.

Suppose a factory requires the previously discussed regression model to best fit their need. There are a lot of model architectures available, a multilayer perceptron (MLP), a CNN, a

U-net, a generative adversarial network (GAN), a YOLO-network etc. Each model has its own number of layers, connections, activation functions, etc. All these networks will have its their own capabilities and thus uncertainty.

Training procedure errors During the training of a model, there are much decisions that need to be made. The learning rate, the number of epochs, the way of using the data, the optimizer used, the weight initialization, etc. Each combination of parameters will result in a different model with its own errors and uncertainty.

The dataset for regression of an object center can be split in different batch sizes, ordered in different ways, etc. The chosen model can be trained with these different ways of handeling data, using different parameters to optimize the chosen model. Again leading to different models who are better in sertain things and worse in other.

Errors by unknown data When the model is fully trained, it can happen that, during inference, the model encounters a situation it has not seen before. This situation was not expected in the original scope of the problem, but the model will need to deal with it, causing uncertainty.

In the factory example, the model is trained for a sertain enviorment and objective: detecting the center of objects in a factory setting. If an object that was not expected to be in the enviorment, a bird for example, enters the camera's viewpoint, it can cause the model to be uncertain on what to do, since it has never seen a bird before. This is not because the enviorment was insufficiently specified, but because of the bird being there unexpectedly.

2.1.3 Sources of uncertainty

In our problem of object localization for robot arm grasping, there are a few sources of uncertainty we want to tackle.

First of all, the aleatoric uncertainty of the data, which will be there because of measurement errors. While we will try to minimize these errors as much as possible, they will still be present in the data. However, as mentioned before, these errors are not the biggest concerns as they could be a way to regulize a model, as long as they are kept subtle.

Secondly, the errors due to the translation from pixel space to real world coordinates. The easiest way to train a model fitted for our task, is to have a dataset containing an image of the object and the corresponding pixel coordinates of the center of the object. However, it should be noted that after the AI has predicted the pixel coordinates of the center of the object, these coordinates need to be translated to real world coordinates for the robot arm to be able to grasp the object. This translation has a lot of opportunities for errors, which can cause a lot of uncertainty in the final output, uncertainty the model does not account for. It is better to train a model which directly outputs the real world coordinates of the center

of the object, as this way the model can learn to output the uncertainty of the translation from pixel space to real world coordinates. This will be further discussed in **??**.

2.2 Quantifying uncertainty

Models who can handle / quantify the uncertainties as described in 1.1 are very sought-after in safety-critical applications like robot arm grasping. For our problem of object localization for robot arm grasping, it is very important to know what factors can cause the model to be uncertain about its predictions and how do we quantify this. When using a model in practice, we are interested in the uncertainty on the prediction of the model, also called predictive uncertainty. This is depended on two things: the uncertainty of the data (data uncertainty), which falls under the aleatoric uncertainty, and the uncertainty of the model (model uncertainty), which falls under the epistemic uncertainty [1].

2.2.1 Aleatoric uncertainty

When we want to quantify the aleatoric uncertainty, we need to capture how mistakes can be made during the capturing and processing of the data. The conventional way of doing this is by using a well generalized model, which is trained on a large dataset. This makes the model learn the general distribution of the data. If the model is not well generalized, it learns the noisy data without real knowledge about the more general distribution of the data. Figure 2.1 shows an example of a model which is not well generalized, and thus captures the noise in the data, leading to a high aleatoric uncertainty.

2.2.2 Epistemic uncertainty

Quantifying epistemic uncertainty is a bit more complex to tackle, as it requires multiple models, hyperparameters, different usage of data etc. to be tested. There are different methods to account for this uncertainty, especially in theory. First of all, we can use a Bayesian approach to model the uncertainty of the model parameters (see 2.3.1). This is a very principled way to model the uncertainty, as it needs an infinite amount of models to capture the uncertainty of the model parameters, which is not possible in practice. Another way to model the uncertainty of the model parameters is by using an ensemble of models (see 2.3.2). This is a more practical way to model the uncertainty, as it is an approximation of the Bayesian approach, using a finite number of models to capture the uncertainty of the model parameters. A third way to model the uncertainty of the model parameters is by using dropout during inference (see 2.3.3). Monte Carlo (MC) dropout, like the ensemble method, is more practical than the Bayesian approach. It keeps dropout turned on during inference, which allows it to capture the uncertainty of the model parameters by generating different outputs for the same input, which can then be combined to get a final output and

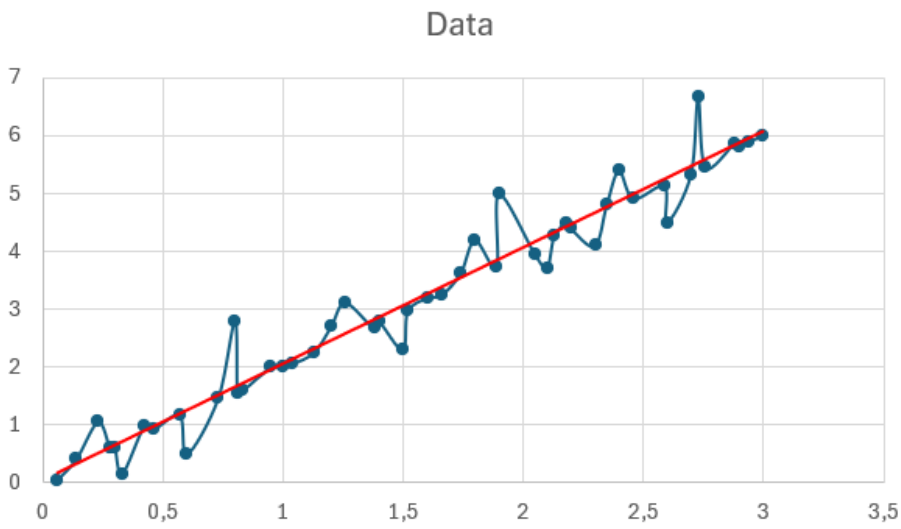

Figure 2.1: The blue line represents a model which is not well generalized, and thus captures the noise in the data. The red line represents a model which is well generalized, and thus captures the general distribution of the data.

uncertainty score.

2.3 Uncertainty-aware AI models

"Conventional" AI models (MLPs, CNNs, GANs, YOLO's, etc.) are not designed to be uncertainty-aware, meaning they do not have a way to quantify the uncertainty of their predictions. This makes that new methods had to be developed to make AI models uncertainty-aware. The ones we will be discussing, all have their foundation in the bayesian way of thinking, which is a very principled, mathematically-sound way to model uncertainty.

2.3.1 Bayesian Neural Networks

A neural network (NN) is a function which maps an input $x \in X$ to an output $y \in Y$ using a model f_θ, as shown in Equation (2.1).

$$y = f_\theta(x) \tag{2.1}$$

In our case, the input vector x is an image of an object, and the output vector y is the predicted coordinates of the center of the object. The parameters θ of the model are learned during the training phase, where the model is trained on a dataset consisting of input-output pairs.

Aleatoric uncertainty [2] For regression problems, the most conventional way to predict targets is to let the model directly predict the targets using Equation (2.1) directly. Its

parameters θ are learned by minimizing a loss function (like the L^2 or L^1 loss) on the training data. However, this does not allow the model to capture the aleatoric uncertainty of the data, as it only predicts a single value for each target, without any indication of how certain it is about this prediction. Instead of predicting a single value for each target, the model can be designed to predict a distribution over the targets as showed by Equation (2.2a). By asuming a Gaussian distribution for the targets, the model can be designed to predict the mean μ and variance σ^2 of the distribution, as shown in Equation (2.2b).

$$p(y|x,\theta) = \mathcal{N}(y; \mu_\theta(x), \sigma^2_\theta(x)), \tag{2.2a}$$

$$f_\theta(x) = [\mu_\theta(x), \log \sigma^2_\theta(x)]^T \tag{2.2b}$$

The variance σ^2 captures the aleatoric uncertainty of the data, while the mean μ captures the prediction of the model. The model can be trained by minimizing the negative log-likelihood of the data under the predicted distribution, as shown in Equation (2.3).

$$min(-\sum_i \log(p(y_i|x_i,\theta))) \tag{2.3}$$

Epistemic uncertainty [2][3] To capture the uncertainty in the model parameters θ, we need the Bayesian approach, otherwise referred to as Bayesian inference. In the Bayesian approach, we place a prior distribution over the model parameters, which captures our beliefs about the parameters before seeing any data. Using the data, we can then compute the posterior distribution over the model parameters θ (given in Equation (2.4)), which captures our updated beliefs about the parameters after seeing the data.

$$p(\theta|X,Y) = \frac{p(Y|X,\theta)p(\theta)}{p(Y|X)} \tag{2.4}$$

$p(\theta)$ is the prior distribution over the model parameters, which can be chosen based on our beliefs about the parameters. When there are no prior beliefs of the model's parameters, its most commenly chosen to use a Gaussian distribution. However, its choise can be a way to regulize the model [4]:

- L^2 regularization is equivalent to using a Gaussian prior.
- L^1 regularization is equivalent to using a Laplace prior.

$p(Y|X,\theta)$ is the likelihood of the data given the model parameters, which captures how well the model fits the data. This is calculated by passing the whole dataset through the model with a softmax output layer and calculating the probability of the data under the predicted distribution.

$p(Y|X)$ is the evidence, which is a normalizing constant that ensures that the posterior distribution is a valid probability distribution. It is calculated by integrating the likelihood over all possible values of the model parameters, as shown in Equation (2.5).

$$p(Y|X) = \int p(Y|X, \theta)p(\theta)d\theta \tag{2.5}$$

However, computing the evidence is not feasible in practice, as it requires integrating over all possible values of the model parameters, of which a DNN has many. As by consequence, the posterior distribution is also not feasible to compute, as it requires computing the evidence. This is why we need to use approximations to compute the posterior distribution.

2.3.2 Deep Ensemble Methods

A deep ensemble method [5] rests on the idea that a combination of (simpler) models can be trained and perform better then a single (complex) model. Not only that, but it also outputs a great uncertainty score about its predictions. A typical ensemble method uses multiple decision trees as their models, a deep ensemble method uses, as the name suggests, uses deep NNs as its models. Creating a deep ensemble method contains 3 main steps:

- A proper scoring rule, which is used as a way to train the models in the ensemble.
- Adversarial training, which is used to make the models in the ensemble more robust.
- The actual ensembling, which combines the outputs of the different models to create a final output and uncertainty score.

The scoring rule A scoring rule is a way to measure the quality of a model's probabilistic predictions. It should encourage the model to generate outputs which are accurate and are capable of telling when it is uncertain about its output. To make this a bit more concrete, in a setting where a model needs to predict if there is a disk in frame or not, a great scoring rule should punish the model hard if it predicts a disk to be in frame with a high probability, when in fact the groud truth states there is no disk in frame. The model should be encouraged to predict a low probability for the disk being in frame, if it is not sure if there is a disk in frame. It should result in a model which is rightfully confident about its predictions when it is sertain about its predictions, while also being able to express its uncertainty when it is not confident in its predictions.

Fortunately, for classification tasks, the loss functions, which are commonly used to train NNs, such as the softmax cross entropy loss, are proper scoring rules.

Adversarial training Adversarial training is a technique which is used to make NNs more robust against small changes who can drastically change the output of the model. Usually these changes are so small a human can not observe the difference, but the model is fooled

to think it is something completely different. For example, when classifying images of disks and cubes, when an image is classified as a disk with a high probability, it could be that with a very small change in pixel value, the network all of a sudden thinks it is a cube with high confidence. This is something we want to avoid at all cost, since this big swing eliminates any notion of uncertainty the model could have had.

Adverserial training tries to fix these big swings by generating new training samples which are very close to some original data (for example a few pixels tweaked) and classified as some other output-class. Using the example, this means it gets an image of a disk, tweaks a few pixels so the model thinks it is a cube, and adds this new image to the training set with the correct label (disk). By doing this, the model learns to be more robust against these small changes, and the big swings in output are reduced.

Ensembling The final step in creating a deep ensemble method is the actual ensembling. The paper [5] suggests to create an ensemble of deep NNs to predict an output with an uncertainty score. Each model in the ensemble should be trained independently, however this does not mean they should be different model architectures. In fact, when randomly initializing the NN parameters, along with shuffeling the training data, the networks where different enough to obtain good results. This makes that no methods like bagging or boosting are needed to create diversity in the ensemble. Which means all the training data can be used for the training of the models in the ensemble, which is benefitial for the deep NNs which need more data to train properly.

To get the final output and uncertainty score, the outputs of the different models are combined. Each model in the ensemble outputs a probability distribution over the different classes. These probability distributions are then averaged to get the final output distribution. The label with the highest probability in this final distribution is taken as the final output with the uncertainty belonging to it. Lets continue the example of classifying images of disks and cubes to make this more concrete. Assuming we have an ensemble of 3 models, and they output the following probability distributions for a given input image:

- Model 1: [0.7 (disk), 0.3 (cube)]
- Model 2: [0.4 (disk), 0.6 (cube)]
- Model 3: [0.9 (disk), 0.1 (cube)]

Averaging these distributions results in the final output distribution: [0.67 (disk), 0.33 (cube)]. The final output is thus 'disk' with an uncertainty score of 0.67.

2.3.3 Monte Carlo dropout

Dropout is a regularization technique commonly used in the training of NNs to prevent overfitting. The idea is to randomly drop some neurons in the network during the training

phase. This forces the model to use all the available neurons and not rely too much on a small subset of them. When the model is fully trained and ready for inference, dropout is usually turned off, using all the neurons to make predictions.

MC dropout [6] expands on this idea by keeping dropout turned on during inference. Every time a set of neurons is dropped, the network changes a bit, which can result in different outputs for the same input. Each alteration of the network can be seen as a different model in an ensemble. By letting the model make multiple forward passes with dropout turned on, we can gather a set of different outputs for the same input with their own predicted probability distributions. These outputs can then be combined in a similar way as the deep ensemble method in Section 2.3.2 to get a final output and uncertainty score.

Reusing the example of classifying images of disks and cubes, assuming we make 3 forward passes with MC dropout, we could get the following outputs:

- Forward pass 1: [0.8 (disk), 0.2 (cube)]
- Forward pass 2: [0.5 (disk), 0.5 (cube)]
- Forward pass 3: [0.6 (disk), 0.4 (cube)]

Averaging these distributions results in the final output distribution: [0.63 (disk), 0.37 (cube)]. The final output is thus 'disk' with an uncertainty score of 0.63.

Chapter 3

Dataset

In this chapter, we will discuss the dataset created for training and evaluating the uncertainty-aware object localization models. We will outline the requirements for the dataset based on the problem statement and objectives defined in Chapter 1. Furthermore, we will describe the data collection process, including the setup and annotation used.

3.1 Setup

The setup we used is shown in Figure 3.1, where we have a realsence camera mounted on a pole in front of a table with a robot arm on it. The view of the camera is a bit wider and longer then the table, but only the part of the table whithin the black boarder lines is usable by the robot.

3.1.1 Realsense camera

Realsense camera's are a type of camera which capture both RGB and depth information, the one we used specificially is the "Realsense Depth Camera D415". The realsense camera's have good support in the form of RealSense SDK [7]. It is a library which supports different languages like Python, C#, ROS, etc. and makes it easy to use the Realsense camera's. We will use its Python binding, which is called "pyrealsense2". It allows us to easily setup and handle both the RGB and depth camera streams independently from each other. It also contains information about the intrinsic parameters of the camera, these embody the focal length and the principal point of the camera. Its importance will become clear in **??**.

3.1.2 Table and robot arm

Table The table is used as support for the the camera, robot arm and objects. As Figure 3.1 shows, the table is equipped with a grid of black lines in the shape of a rectangle and a black square. Each line is 2cm apart from each other and is used as a reference so we can capture

Figure 3.1: The setup used.

the real-world coordinates of the objects. The black square is used as the origin of this maden coordinate system, more specifically the bottom right corner of the square is the origin. The x-axis goes horizontally (relative to the camera's point of view (PoV)) to the right, the y-axis goes vertically downwards. There is no z-axis defined as all the objects are roughly the same height; consequently, the robot arm only needs to know the x and y coordinates of the objects to be able to grasp them. More information about the specific use of the grid will be given in **??**.

Robot arm The robot arm used in our setup is the "ABB IRB 1200" [8]. It is equipped with a vacuum gripper, which is used to grasp the objects on the table. The robot arm is controlled by a computer, which sends commands to the robot arm to move to specific coordinates and to open and close the gripper. Previous work with this specific robot arm [9] will be used to control it.

3.2 Labeling

We needed to create a dataset for regression of the position of the objects on the table. There where three approaches we could take to do this:

- We could use the pixel coordinates.
- We could use the real-world coordinates.
- We could use the robot's position.

There are some advantages and problems with all three approaches.

3.2.1 Pixel coordinates

Pixel coordinates are coordinates in the image plane of the camera, they are defined by the number of pixels in the x and y direction. A computer sees an image as a 2D array of pixels per colour channel, so for RGB images, there are 3 2D arrays in total, one for each colour channel. Conventionally, the origin of the pixel coordinates is the first value in the array ($img_r[0][0]$), the x-axis goes horizontally to the right and the y-axis goes vertically downwards. The libraries most commonly used for computer vision, like OpenCV [10] and PyTorch [11], use pixel coordinates for their operations on images, which makes it very convenient to use and incooperate. It makes it easy to annotate the dataset, as we can create a simple script to click on the objects in the images and save the pixel coordinates of the clicks as annotations, as shown in Figure 3.2. Even more, using some of the tools provided by the OpenCV [10] library, we can even automate this clicking process. Its resulting annotation should be more accurate and consistent than a user clicking on the center of the

Figure 3.2: Annotating the pixel coordinates of the objects.

object, purely visually. However, it is not 100% consistent, as some objects were not detected and thus still required some manual clicking.

This way of annotating in pixel space, requires some post-processing to achive what we want, which is the real-world coordinates of the object so the robot arm can use them to grasp the objects. This transformation requires information about the characteristics of the camera, called the intrinsic parameters, as well as the position and orientation of the camera relative to the table, which are called the extrinsic parameters. The intrinsic parameters of the camera we are using, are provided by the Realsense SDK [7] and should be very accurate, as they are measured and provided by the manufacturer. The extrinsic parameters need to be calibrated for each specific setup, they consist of a rotation and translation from the camera's coordinate system (pixel coordinates) to the table's coordinate system. OpenCV [10] provides a convenient way to do this calibration, this being the "findChessboardCorners" function. It detects the corners of a chessboard pattern, as shown in Figure 3.3, these corners can then be used to calculate the rotation and translation vector using the "solvePnP" function, which is also provided by OpenCV [10]. Once the parameters are obtained, we can use them to transform the pixel coordinates of the objects to real-world coordinates. However, this

Figure 3.3: The chessboard pattern used for calibration.

process is not perfect and can introduce some errors, which can be a source of uncertainty in the dataset, which is something we want to avoid as much as possible.

3.2.2 Real-world coordinates

We presume real-world coordinates as the coordinates measured in the physical world, in our case they are relative to an origin defined on the table. Since this way of doing the coordinates uses real measurements, it is more intuitive and directly usable for the robot arm. However, it comes with an important caveat, as it is more difficult to annotate data efficiently. We need to define an origin for the coordinate system, as well as the x and y axis, and then we need to measure the coordinates of the objects relative to this origin. This measuring can be done with a ruler, but this will be too time-consuming and cause too much human error, which can introduce uncertainty in the dataset which we want to minimize. A better option is to use a predefined grid on the table, which we could detect and use to measure the coordinates of the objects. This is better, but we can not draw a grid on the table, as we would like the AI to do inference without any outside assistance (like a grid).

We thus need a way to create a "virtual" grid which can easily be removed from the images, but can still be used to measure the coordinates of the objects. This is where the black lines around the table come in.

They could be detected by the camera [12] with the help of the OpenCV [10] library and used to create this virtual grid by connecting opposite lines. However, this did not work well, as not all the lines were detected. This is most probably due to the lighting not being consistent enough for the filters used to detect the lines to work properly. An other problem is the resolution of the camera being too low to detect the individual lines, this was most prominent in the corners of the table where the lines are closest to each other. We ended up using a more manual approach, where we manually annotated the pixel coordinates of the center of the lines as well as the origin square. This introduces some human error, but still ensured we got a consistent and accurate enough grid to measure the real-world coordinates of the objects.

All this setting up of the grid, is done virtually before any labeling is done and saved as a reference for the labeling process. When using the system to either label the dataset or to do inference, the image is cropped so the black lines are not visible anymore. Now we have a way to measure the real-world coordinates of the objects without introducing too much human error, while still being able to remove the grid from the images so the AI can do inference without any outside assistance.

3.2.3 Robot's position

Ultimately, the robot arm needs to move itself to the correct position to be able to grasp the objects. This means that we could also use the robot's position as labels for the dataset. However, this, just as the real-world coordinates, is not that easy to annotate, as we need to move the robot arm to the correct position and then read out its coordinates. There is no easy trick (like using OpenCV [10] for real-world coordinates) to do this fast, accurately but most of all consistent, so we would need to do this manually for each data point.

3.3 Capturing and using data

3.3.1 Collecting data

To actually capture the data, we created a script which implements both the pixel-coordinates and real-world coordinates approach, this so we can choose which one to ultimately use for our models. It is an interactive script, which shows the RGB and depth images from the camera in real-time and allows us to capture the data by pressing a button. When the button is pressed, the script tries to automatically detect the center of the object. The user can then either accept the detected coordinates or manually click on the center of the object to correct the coordinates. After pinpointing the object's center, the script crops the

```
{
    "pixel": {
        "x": 221,
        "y": 62
    },
    "world": {
        "x": 288.09246369430474,
        "y": 134.27617670846834
    }
}
```

Listing 1: Example of the JSON file structure.

image so the calibration lines are not visible anymore and saves the RGB and depth images. Following this, it calculates the real-world coordinates of the object and saves them along the pixel-coordinates in a JSON file, which is structured as shown in Listing 1.

Each trio of RGB image, depth image and JSON file is considered one data point in the dataset, all with a unique identifier.

3.3.2 PyTorch Dataset

Since we are using the PyTorch [11] framework for the training and evaluation of our models, we needed a way to easily load and preprocess the data in a way that is compatible with it. PyTorch [11] provides a "Dataset" class, which is an abstract class that we can inherit from and implement our own dataset.

We kept two options open when creating this class, as we wanted to keep as much options open as possible to differentiate between the different approaches.

- Have a 3D tensor as input, which is the RGB image, or have a 4D tensor as input, which is the RGB image concatenated with the depth image.
- Use the pixel coordinates as labels, or use the real-world coordinates as labels.

When looking at the depth images (of which an example is shown in Figure 3.4) captured by the Realsense camera, we noticed that they did not contain that much information about the object, as the objects are pretty small in height.

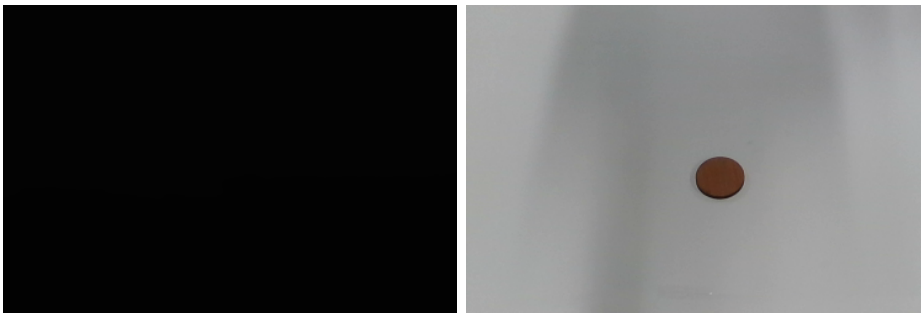

Figure 3.4: An example of a depth image with its corresponding RGB image captured by the Realsense camera.

Chapter 4

Models for uncertainty aware object detection

In this chapter, we will discuss the different uncertainty-aware models discussed in Chapter 2 and compare their adaptability to a real-world scenario.

Chapter 5

Safety mechanisms for uncertain predictions

In this chapter, we will explore various safety mechanisms that can be implemented to turn the outputs of an uncertainty-aware object detection model into safe and reliable actions for a robot arm.

Chapter 6

Conclusion

6.1 Future work

Pixel-coordinates vs real-world coordinates, which more accuratly represents the real-world position of the objects?

Bibliography

[1] J. Gawlikowski, C. R. N. Tassi, M. Ali, J. Lee, M. Humt, J. Feng, A. Kruspe, R. Triebel, P. Jung, R. Roscher, M. Shahzad, W. Yang, R. Bamler, and X. X. Zhu, "A survey of uncertainty in deep neural networks", *Artificial Intelligence Review*, vol. 56, no. 1, pp. 1513–1589, Oct. 2023, issn: 1573-7462. doi: 10.1007/s10462-023-10562-9. [Online]. Available: https://doi.org/10.1007/s10462-023-10562-9.

[2] F. K. Gustafsson, M. Danelljan, and T. B. Schon, "Evaluating scalable bayesian deep learning methods for robust computer vision", in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, Jun. 2020.

[3] G. Li, L. Yang, C.-G. Lee, X. Wang, and M. Rong, "A bayesian deep learning rul framework integrating epistemic and aleatoric uncertainties", *IEEE Transactions on Industrial Electronics*, vol. 68, no. 9, pp. 8829–8841, 2020.

[4] M. Verbeke, *Artificial neural networks and deep learning, Part 5: Bayesian learning of neural networks*, Lecture notes, KU Leuven, Course lecture, 2025.

[5] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles", in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30, Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/9ef2ed4b7fd2c810847ffa5fa85bce38-Paper.pdf.

[6] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning", in *Proceedings of The 33rd International Conference on Machine Learning*, M. F. Balcan and K. Q. Weinberger, Eds., ser. Proceedings of Machine Learning Research, vol. 48, New York, New York, USA: PMLR, 2016, pp. 1050–1059. [Online]. Available: https://proceedings.mlr.press/v48/gal16.html.

[7] Eran, S. Dorodnicov, N. Azkiel, remibettan, aangerma, S. Orsten, O. Meir, ev-mp, Noha Yassin, D. Diakopoulos, Z. Shahaf, A. Avraham, M. Katz, I. Carpis, noacoohen, Tamir, MMirbach, S. Khshiboun, doronhi, AnnaRomanov, gwen2018, J. Radford, Noy-Zini, alowenst01, A. Bernstein, D. Tong, B. Fulkerson, T. Shao, gilpazintel, and shiritbrook, *Realsenseai/librealsense*, https://github.com/realsenseai/librealsense, Mar. 2026. [Online]. Available: https://github.com/realsenseai/librealsense.

[8] ABB Robotics, *Product specification: Irb 1200 (on omnicore)*, Document ID: 3HAC081417-001, Revision: L, ABB Robotics Sweden AB, S-721 71 VÄSTERÅS, Sweden, Nov. 2025. [Online]. Available: https://new.abb.com/products/robotics.

[9] M. De Ryck, *Robot_demonstrator: Software for the robot demonstrator at ku leuven bruges*, https://github.com/MatthiasDR96/robot_demonstrator, ABB IRB1200 robot control with RealSense D415 and SICK Microscan 3, Dec. 2024.

[10] G. Bradski, *The OpenCV Library*, 2000.

[11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library", in *Advances in Neural Information Processing Systems 32*, Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: `http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf`.

[12] S. O. Contributors, *How to detect lines in OpenCV?*, `https://stackoverflow.com/questions/45322630/how-to-detect-lines-in-opencv`, Online; accessed 15-April-2026, 2017.

Chapter 7

GenAI code of conduct for students

Generative AI (GenAI) assistance tools can be used to generate text, image, code, video, music, or combinations of these. It includes typical tools like (but this list is not limited to): ChatGPT, Google Gemini, MS Copilot, Midjourney, Claude.ai, Perplexity.ai, Dall-E, etc.

Student Information

Student name: Thijs Alens

This form is related to my Master thesis.
Title Master thesis: Modeling object localization uncertainty for robot grasping Promoter: _____
Daily supervisor: _____

Use of GenAI Assistance

Please indicate with "X" (possibly multiple times) in which way you were using GenAI:

- ☐ I did not use any GenAI assistance tool.
- ☐ I did use GenAI assistance. In this case, specify which ones (e.g., ChatGPT, ...): _____

GenAI Assistance Used As/For

- ☐ MS Copilot

☐ Language assistance

☐ Search engine

☐ Literature search

☐ Short-form input assistance

☐ Generating programming code

☐ Generating new research ideas

☐ Generating blocks of text

☐ Other (specify): _____

Code of Conduct for Different Uses

Language Assistant for Reviewing or Improving Texts

This use is similar to using spelling and grammar check tools, and you do not have to refer to the use of GenAI in the text. Be careful:

- Using GenAI tools on texts you did not write yourself to cover up plagiarism (by paraphrasing original texts) is not allowed.

Search Engine for Information or Existing Research

This use is similar to a Google search or checking Wikipedia. If you then write your own text based on this information, you do not have to refer to the use of GenAI in the text. Be careful:

- Be aware that the output of the GenAI tool cannot be guaranteed as a 100% reliable source of information.
- The output may not be entirely correct and is limited due to the databases it uses. Knowledge evolves and may change over time, and it may be that the database of the GenAI tool is not up to date.

Literature Search

This use is comparable to a Google Scholar search. Be careful:

- Be aware that the output is restricted to the database it is built on. After this initial search, look for scientific sources and conduct your own analysis.
- GenAI tools (like ChatGPT) may output no or wrong references. As a student, you are responsible for further checking and verifying the accuracy of references.

Short-form Input Assistance

This use is similar to Google Docs powered by generative language models.

Generating Programming Code

The use of GenAI for coding should be explicitly allowed by the teacher. If used for coding, correctly mention the use of GenAI assistance and cite it.

Generating New Research Ideas

Further verify whether the idea is novel or not. It is likely that it is related to existing work, which should be referenced.

Generating Blocks of Text

Inserting blocks of text without quotes and a reference to GenAI assistance in your report or thesis is not allowed. Be careful:

- When you literally copy elements from a conversation with a GenAI tool, quote them between quotation marks and refer to them according to the specified reference style.
- Describe the use of the GenAI tool (tool name, version, date, etc.) in the method section and optionally add the full conversation as an attachment.

Other

Contact the professor of the course or the promoter of the thesis. Inform also the program director. Motivate how you comply with Article 84 of the exam regulations. Explain the use and added value of ChatGPT or another AI tool.

Further Important Guidelines and Remarks

- GenAI assistance cannot be used related to data or subjects under Non-Disclosure Agreement.
- GenAI assistance cannot be used related to sensitive or personal data due to privacy issues.
- Take a scientific and critical attitude when interacting with GenAI assistance and interpreting its output.
- As a student, you are responsible for complying with Article 84 of the exam regulations: your report or thesis should reflect your own knowledge, understanding, and skills.

Exam Regulations Article 84

"Every conduct individual students display with which they (partially) inhibit or attempt to inhibit a correct judgement of their own knowledge, understanding and/or skills or those of other students, is considered an irregularity which may result in a suitable penalty. A special type of irregularity is plagiarism, i.e., copying the work (ideas, texts, structures, designs, images, plans, codes, . . .) of others or prior personal work in an exact or slightly modified way without adequately acknowledging the sources."

Additional Reading

More information about being transparent on the use of GenAI assistance and about citing and referencing GenAI can be found on the student website.

A Few Final Words

If you are uncertain whether or not you should declare your use of AI tools, discuss the matter with your teacher or promoter. It is safer to declare AI use when it is not needed than to withhold that declaration when it is required.

Finally, remember that advanced AI tools are new and that they can do things they could not do until recently. It is important to follow up on the most recent developments in AI technologies and communicate openly with your teachers, assistants, supervisors, and peers.